



**Nombre de la institución:**

**Materia: Redes**

**Profesor: Oscar Obregón**

**Documentación técnica del desarrollo**

**Alumno: Ocampo Joaquin**

**Curso: 5°1ra**

## **1. Para qué sirve el programa**

EscanerMVC es una aplicación de escritorio desarrollada en Java diseñada para realizar operaciones de escaneo y diagnóstico en redes locales o remotas. Su propósito principal es facilitar a los usuarios la comprobación de conectividad, el análisis de direcciones IP y la gestión de resultados de forma visual y organizada.

La herramienta centraliza el flujo de red bajo una arquitectura limpia, permitiendo validar entradas de IP, ejecutar tareas de sondeo en segundo plano para no bloquear la interfaz y presentar los hallazgos en tiempo real al operador.

## **2. Cómo está armado el sistema (Arquitectura y Diagramas)**

El sistema se diseñó siguiendo estrictamente el patrón de arquitectura Modelo-Vista-Controlador (MVC), lo que garantiza una separación clara de responsabilidades:

- **Modelo (modelo/):** Contiene la lógica de negocio, manejo de datos, utilidades de red y validaciones (EscaneoService, ModeloResultados, ResultadoEscaneo, ValidadorIP, IPUtils).

- Vista (vista/): Contiene los componentes gráficos de la interfaz de usuario (VentanaPrincipal).
- Controlador (controlador/): Actúa como intermediario entre la vista y el modelo (EscaneoController).
- Punto de Entrada (Main.java): Inicializa los componentes de la aplicación.

### 3. Qué métodos se usaron y por qué

Para construir la lógica funcional del sistema, se implementaron clases y métodos especializados:

- ValidadorIP.java e IPUtils.java: Contienen métodos de validación de sintaxis y manipulación de cadenas para las direcciones IP. *¿Por qué?* Para prevenir errores de ejecución y asegurar que el usuario ingrese formatos de red válidos antes de disparar peticiones.
- EscaneoService.java: Agrupa los métodos encargados de la lógica de sondeo y comunicación de red. *¿Por qué?* Aislar las operaciones de red en un servicio dedicado evita acoplar la interfaz gráfica con los protocolos de conectividad.
- ModeloResultados.java y ResultadoEscaneo.java: Gestionan las estructuras de datos que almacenan el estado de cada IP escaneada (activa, inactiva, latencia, etc.). *¿Por qué?* Permiten estructurar la información para que pueda ser enlazada directamente con los componentes visuales de tablas o listas en la interfaz.
- EscaneoController.java: Conecta los clics de los botones y eventos de la VentanaPrincipal con la ejecución de los servicios del modelo. *¿Por qué?* Evita código espagueti dentro de las clases visuales y centraliza el manejo de eventos.

### 5. Qué problemas aparecieron y cómo se solucionaron

1. Problema: Congelamiento de la interfaz gráfica (UI Freeze) al realizar los escaneos de red.
  - Solución: Se implementó la ejecución de los procesos de red en hilos secundarios (Threads / tareas concurrentes) dentro de EscaneoService, asegurando que el hilo principal de la interfaz gráfica (VentanaPrincipal)[cite: 1] permanezca fluido y responsivo para el usuario.
2. Problema: Excepciones por entradas de texto vacías o mal formateadas en las direcciones IP.
  - Solución: Se desarrolló la clase ValidadorIP[cite: 1] para filtrar de manera estricta los datos antes de enviarlos al controlador y al servicio de escaneo, mostrando alertas tempranas al usuario.

